

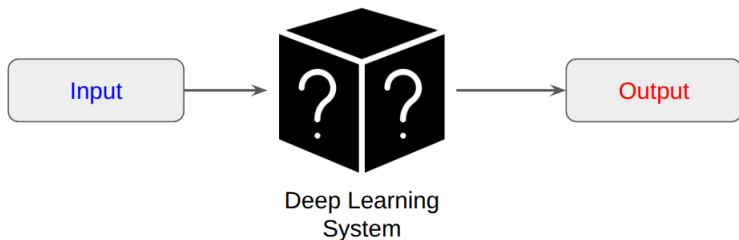
Deep Learning

A brief review of the essentials

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Why prerequisites matter?

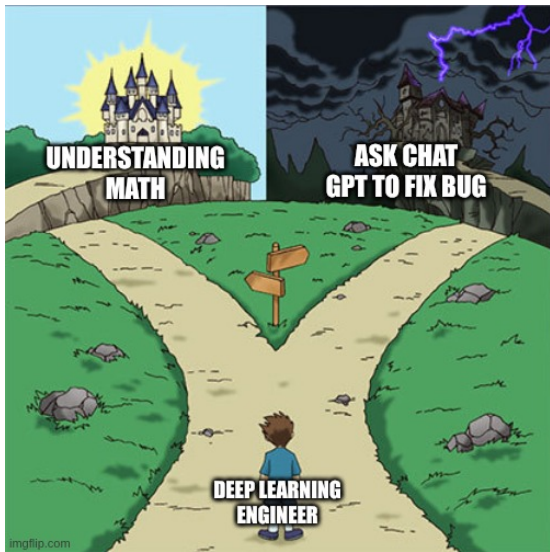


- Deep Learning builds on [math](#) and [probability](#) foundations.
- Without them most of the course may feel like a "black-box" to you.
- With them you will have more confidence and the rest of the course will become much easier to follow.

How the prerequisites fit in Deep Learning?

- **Linear algebra:** data representations and transformation
- **Calculus:** training neural networks via backpropagation
- **Probability:** prediction of the model and generalization
- **Optimization:** finding the best model parameters

Thus, we will review these essentials first so that you do not need to worry about them later.



Lecture overview

1. Linear Algebra Basics

2. Calculus Refresher

3. Probability Theory

4. Optimization

Linear Algebra Basics

Scalars

- In DL we will be manipulating numbers. We call these values as **scalars**.
- Example: Today the temperature is 28 degree celsius. The number "28" is a **scalar** value.
- Mathematically, we denote them using lowercase letters, eg, x , a , z .
- If we use the following notation:

$$y \in \{0, 1\},$$

it means the scalar y takes on values either 0 or 1.

- Sometimes you will see notations like:

$$z \in \mathbb{R},$$

which means z is a real-valued scalar. Meaning, it can take on any numerical value between negative infinity and positive infinity. The symbol $\in$ means "*is an element of*".

Vectors

- **Vector** can be thought as an array of scalars. Typically in DL we will be working with vectors.
- Mathematically, we denote vectors using bold lower case letters: $\mathbf{x}, \mathbf{y}, \mathbf{a}$.
- By default, a vector of length n is written in column format:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix},$$

Here, $x_1, x_2, \dots, x_n$ are the elements of the vector. The i^{th} element x_i is a **scalar** value.

- To indicate that a vector contains n elements (or n -dimensional), we write $\mathbf{x} \in \mathbb{R}^n$. For example, if there are 512 elements in $\mathbf{x}$, we will write $\mathbf{x} \in \mathbb{R}^{512}$.

Matrices

- A **matrix** is a rectangular array or a grid of numbers.
- Mathematically, we denote matrices by bold capital letters: **A**, **X**, **Y**.
- A matrix with m rows and n columns is written in the following way:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix},$$

where a_{ij} is a scalar value that belongs to **A**'s i^{th} row and j^{th} column.

- We use the expression $\mathbf{A} \in \mathbb{R}^{m \times n}$ to indicate that a matrix contains $m \times n$ scalars, arranged in m rows and n columns.

Tensors

- **Tensors** are multidimensional arrays of numbers, which generalize scalars (0 dimensions), vectors (1 dimension), and matrices (2 dimensions) to any number of dimensions.
- We will denote tensors by capital letters: A, B, X .
- For example, we use the expression $A \in \mathbb{R}^{m \times n \times p}$ to indicate that the tensor contains $m \times n \times p$ scalar values.
- Note, you can have any arbitrary number of dimensions in tensors, and not just three.
- All throughout DL, we will mostly work with tensors.

Dot Products

- One of the most fundamental operations in DL is **dot product**.
- Given two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ of dimensions d , their dot product $\mathbf{x}^\top \mathbf{y}$ (also called *inner product* $\langle \mathbf{x}, \mathbf{y} \rangle$) is given as:

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^d x_i y_i.$$

- In simple words, the above equation performs **element-wise multiplication** of the scalar values of the two vectors, followed by summing the products obtained from d such multiplications.
- The outcome of performing a dot product is a scalar value.
- Intuitively, dot product between two vectors tells us how similar are two vectors:
 - ▶ Higher value indicates higher similarity, and vice versa.

Matrix-Vector Products

- Similarly to the dot product between two vectors, we can take a product between an $m \times n$ matrix $\mathbf{A}$ and n -dimensional vector $\mathbf{x}$.
- We denote matrix-vector product as $\mathbf{Ax}$. The output will be another vector of dimension m .
- You can think of this operation as: performing m dot products between each row of $\mathbf{A}$ and $\mathbf{x}$.
- To be able to do the product we need to ensure that the dimensions are compatible:
 - ▶ The column dimension of $\mathbf{A}$ should be equal to the dimension of $\mathbf{x}$.

Matrix-Matrix Products

- You can easily generalize this idea to the **matrix-matrix** product.
- Let's say you have two matrices $\mathbf{A} \in \mathbb{R}^{n \times k}$ and $\mathbf{B} \in \mathbb{R}^{k \times m}$. Their product is given as:

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{n \times m}$$

- We can think of matrix-matrix product $\mathbf{AB}$ as performing $m \times n$ dot products, each operation producing one scalar value, followed by collecting the results in a $n \times m$ matrix.

Norms

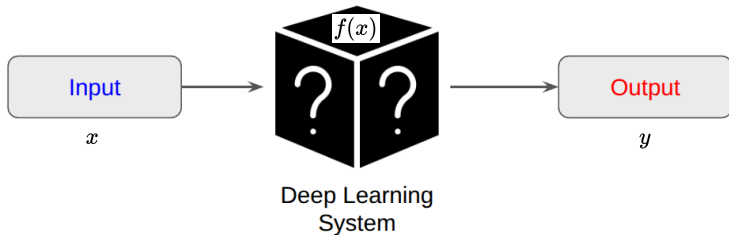
- **Norm** of a vector tells us how *big* is the vector.
- The most commonly used norm is the ℓ_2 norm, which measures the Euclidean length of a vector.
- Formally, the ℓ_2 norm of a vector $\mathbf{x} \in \mathbb{R}^d$ is expressed as:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^d x_i^2}.$$

- Another way to interpret ℓ_2 norm is taking the dot-product of a vector with itself, followed by the square root operation.
- There are other norms such as ℓ_1 norm and ℓ_∞ norm, which we will discuss later in the course.

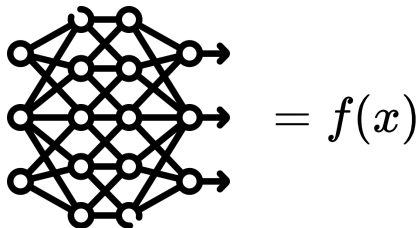
Calculus Refresher

Functions



- A DL system under the hood is nothing other than a **function**.
- A function is simply mapping inputs to outputs.
- For example, if you have a function that adds 3 to any number, the mathematical representation is $y = f(x) = x + 3$. If you input 2, the output is $y = f(2) = 2 + 3 = 5$.

Functions



- The neural network can be viewed as a function.
- Our goal in DL is to "learn" this function $f(x)$ from examples (or data). Thus, it becomes an optimization problem, where we are looking for the best function $f(x)$ that explains the data.
- Calculus is a key prerequisite in solving the optimization problem.
- Hence, we will look at some key concepts in calculus.

Derivatives and Differentiation

- **Derivative** is the rate of change in a function with respect to changes in its arguments.
- Often times, we want to measure how much the function output changes with a small change in the value of its "internal parameters". For this reason, derivatives are useful.
- Formally, the derivative of f at a point x is defined as:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- Since, we need to compute derivatives, it is important to choose functions inside the neural network that are differentiable (more on this later).

Derivatives and Differentiation: An example

- Let's consider a simple example. Let's define $u = f(x) = 3x^2 - 4x$. This is the equation of a parabola.
- We want to find the derivative of $f(x)$.
- The derivative of $f(x)$ is $\frac{d}{dx}(3x^2 - 4x) = \frac{d}{dx}(3x^2) - \frac{d}{dx}(4x) = 6x - 4$.
- This is nothing but an equation of a straight line: $y = 6x - 4$.

Partial Derivatives

- In DL, we need to work with functions of many variables and not just one variable.
- Let $y = f(x_1, x_2, \dots, x_n)$ be a **multivariate** function of n variables. When we have many variables what we compute are called as **partial derivatives**.
- A term $\frac{\partial y}{\partial x_i}$ means the partial derivative of y with respect to x_i . We compute this by treating all the other variables, ie, $x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n$ as constants.
- For example, $y = f(x_1, x_2) = 4x_1^2 - 6x_2$ is a function of two variables: x_1 and x_2 . The partial derivative of y with respect to x_1 is given as: $\frac{\partial y}{\partial x_1} = 8x_1$.
- Similarly, you can compute the partial derivative of y with respect to x_2 .

Partial Derivatives and Gradients

- We can concatenate partial derivatives of a multivariate function with respect to all its variables to obtain a vector that is called the **gradient** of the function. It is denoted as $\nabla_{\mathbf{x}}f(\mathbf{x})$.
- In the above example, the gradient of the function f with respect to $\mathbf{x}$ is a vector of 2 partial derivatives:

$$\nabla_{\mathbf{x}}f(\mathbf{x}) = \begin{bmatrix} 8x_1 \\ -6 \end{bmatrix}$$

- Generalizing to n variables, the gradient of the function f with respect to $\mathbf{x}$ is a vector of n partial derivatives:

$$\nabla_{\mathbf{x}}f(\mathbf{x}) = \left[\frac{\partial}{\partial x_1} f(\mathbf{x}), \frac{\partial}{\partial x_2} f(\mathbf{x}), \dots, \frac{\partial}{\partial x_n} f(\mathbf{x}) \right]^\top$$

Chain Rule

- A **nested function** means applying one function inside another: $y = f(g(x))$.
- In DL, neural networks are essentially many nested functions stacked together.
- For example, if $g(x) = 2x$ and $f(x) = x^2$, then $y = f(g(x)) = (2x)^2 = 4x^2$.
- Gradients are difficult to calculate when we have nested functions (of functions (of functions...)). In such cases **chain rule** is handy.
- For the above example of $y = f(g(x))$, let $y = f(u)$ and $u = g(x)$. We want to find $\frac{dy}{dx}$.
- We will apply chain rule to find $\frac{dy}{dx}$:

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx} = \frac{d}{du} f(u) \frac{d}{dx} g(x)$$

- The chain rule of differentiation is what makes backpropagation so efficient and consequently training deep neural networks.

Chain Rule: An example

- Let $y = f(u) = u^2$ and $u = f(x_1, x_2) = x_1 + x_2$. We want to find how much y varies with respect to both x_1 and x_2 , ie, we are after the gradient: $\left[\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}\right]$.
- **Step 1:** find the partial derivatives of u with respect to (w.r.t) x_1 and x_2 .

$$\begin{bmatrix} \frac{\partial u}{\partial x_1} \\ \frac{\partial u}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f(x_1, x_2) \\ \frac{\partial}{\partial x_2} f(x_1, x_2) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} (x_1 + x_2) \\ \frac{\partial}{\partial x_2} (x_1 + x_2) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- **Step 2:** find the derivative of y w.r.t u : $\frac{dy}{du} = \frac{d}{du}(u^2) = 2u$.
- **Step 3:** apply chain rule to get:

$$\nabla_{\mathbf{x}} y = \begin{bmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \end{bmatrix} = \frac{dy}{du} \begin{bmatrix} \frac{\partial u}{\partial x_1} \\ \frac{\partial u}{\partial x_2} \end{bmatrix} = 2u \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2u \\ 2u \end{bmatrix} = \begin{bmatrix} 2(x_1 + x_2) \\ 2(x_1 + x_2) \end{bmatrix}$$

Probability Theory

Probability

- Many events can not be predicted with total certainty. The best we can say is how likely they are to happen, using the idea of **probability**.



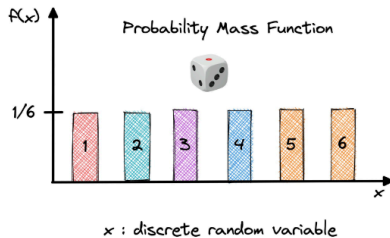
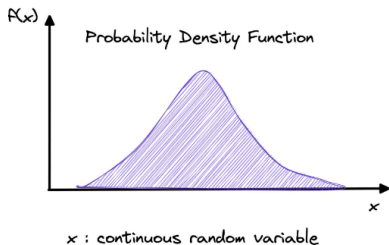
- Intuitively, probability expresses a degree of belief.
- Similarly, in DL we deal with uncertain quantities and sometimes stochastic (nondeterministic) quantities. Therefore, we use probability theory in DL.

Random Variables

- A **random variable** (r.v.) is a variable that can take on different values randomly.
- Let's say we are tossing a coin. We denote the random variable with any letter, such as X or Y .
- In the case of the coin toss, the random variable X can take two values: either $X = \text{head}$ or $X = \text{tail}$.
- More formally, we write r.v taking on a value as $X = x$, where the capital letter denotes the r.v. and the lower case letter denotes the **actual value** of the r.v.
- A r.v. can be either discrete or continuous
 - ▶ A discrete r.v. is one that has a finite or countably infinite number of states (eg, throw of a die or coin toss).
 - ▶ A continuous r.v is associated with a real value (eg, water pressure or humidity).

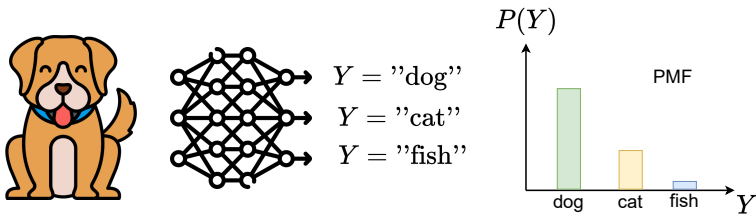
Probability Distributions

- A **probability distribution** is a description of how likely a random variable or set of random variables is to take on each of its possible states.
- Depending on the nature of the r.v., the probability distribution also varies.
 - ▶ For discrete r.v. we have **probability mass function** (PMF)
 - ▶ For continuous r.v. we have **probability density function** (PDF)



Probability Mass Function

- A probability distribution over discrete variables is described using a **probability mass function** (PMF). We typically denote PMFs with a capital P .



- The probability that $Y = y$ is denoted as $P(y)$, with a probability of 1 indicating that $Y = y$ is certain, and a probability of 0 indicating that $Y = y$ is impossible.
- In the above example, a high value of $P(Y = \text{"dog"})$ means that the input image is more likely to be a dog than a cat or a fish.

Probability Mass Function

- A PMF must satisfy the two key properties:
 - ▶ $P(x)$ will always lie between 0 and 1. Formally, we write as $\forall x \in X, 0 \leq P(x) \leq 1$.
 - ▶ The sum of probabilities is equal to 1. Formally, $\sum_{x \in X} P(x) = 1$.
- When a discrete r.v. can take on k states (or values) with equal probability, we call it a **uniform distribution**. Example: rolling of a die, or fair coin toss.

$$P(X = x_i) = \frac{1}{k}$$

- In DL, we will be using PMFs more than PDFs.

Conditional Probability

- In many cases, we are interested in the probability of some event, given that some other event has happened. This is called **conditional probability**.
- We denote the conditional probability that $Y = y$ given $X = x$ as $P(Y = y \mid X = x)$. It can be computed with the formula:

$$P(Y = y \mid X = x) = \frac{\overbrace{P(Y = y, X = x)}^{\text{joint probability}}}{\underbrace{P(X = x)}_{\text{marginal probability}}},$$

where $P(Y = y, X = x)$ refers to the **joint probability** that $X = x$ and $Y = y$ **simultaneously**.

- In the previous example of the animal classifier, we are actually computing a conditional probability $P(Y = \text{'dog'} \mid X = x)$, where x is the input image.

Chain Rule of Conditional Probabilities

- Any joint probability distribution over many random variables may be **decomposed** into conditional distributions over only one variable.
- Let's say we have a joint distribution over n variables: $P(X^{(1)}, X^{(2)}, \dots, X^{(n)})$. This can be decomposed as:

$$P(X^{(1)}, X^{(2)}, \dots, X^{(n)}) = P(X^{(1)})P(X^{(2)} | X^{(1)}) \dots P(X^{(n)} | X^{(1)} \dots X^{(n-1)})$$

- This is used for modelling sequences of text in Large Language Models (LLMs), where we want to estimate the probability of the sentence "The cat yawned" as:

$$P(\text{"The cat yawned"}) = P(\text{"The"})P(\text{"cat"} | \text{"The"})P(\text{"yawned"} | \text{"The", "cat"})$$

- This decomposition technique is known as the **chain rule** or **product rule** of probability.

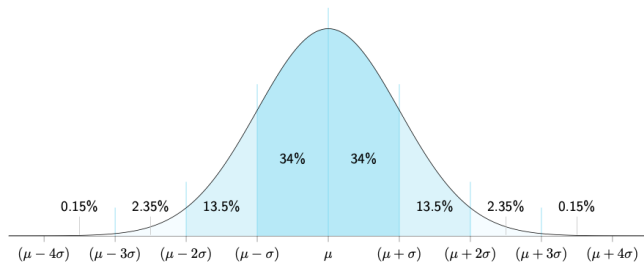
Common Probability Distributions

- **Bernoulli Distribution.** It is a distribution over a single binary r.v. It is controlled by a single parameter $\phi \in [0, 1]$.

$$P(X = 1) = \phi, \text{ and } P(X = 0) = 1 - \phi$$

- ▶ It is used in situations where the decision is binary: cat vs not cat, head vs tail, etc.
- **Multinoulli or Categorical Distribution.** It is a distribution over a single discrete r.v. with k different states. It is controlled by a vector $\mathbf{p} \in [0, 1]^{k-1}$, where p_i is a scalar value and gives the probability of the i -th state.
 - ▶ This is used when a neural network predicts over a set of categories of animals (previously seen example).

Common Probability Distribution



- The most commonly used distribution over real numbers is the **normal distribution** or the **Gaussian distribution**.
- In **Gaussian Distribution** most data points gather around an average value, with a tendency for values to decrease as they move away from the mean.
- It has two parameters: **mean** μ and **standard deviation** σ .
- If the $\mu = 0, \sigma = 1$ we call it **standard** normal distribution.

Common Probability Distribution

- In the d -dimensional case we call it **multivariate** normal distribution.
- It is parameterized by: **mean vector** $\mu \in \mathbb{R}^d$ and **covariance matrix** $\Sigma \in \mathbb{R}^{d \times d}$.
- Also written as $\mathcal{N}(\mu, \Sigma)$, its density is given as:

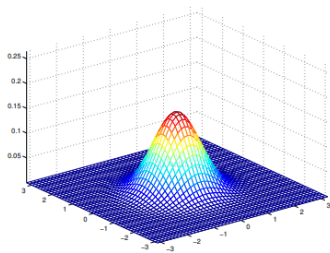
$$p(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right),$$

where $|\Sigma|$ is the determinant of Σ .

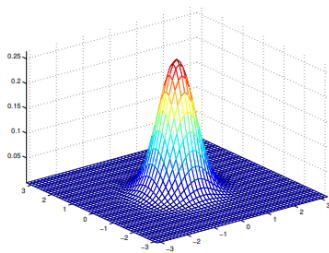
- A Gaussian with $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ is called **standard normal distribution**.

Common Probability Distribution

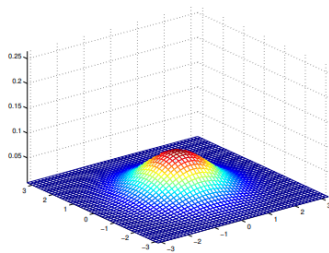
- Examples of multivariate normal distributions with zero mean $\mu \in \mathbb{R}^{2 \times 1}$ and covariance matrices that are a scalar multiple of 2×2 identity matrix I .



$$\mathcal{N}(\mu, I)$$



$$\mathcal{N}(\mu, 0.6I)$$

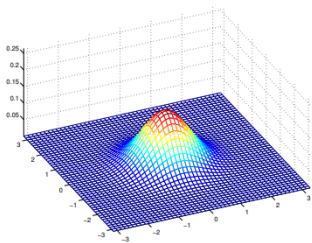


$$\mathcal{N}(\mu, 2I)$$

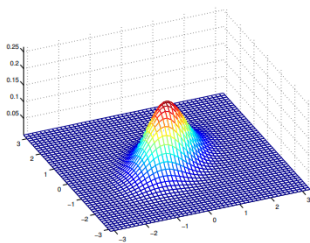
- When the covariance matrix is a scalar multiple of I , it is called **isotropic Gaussian distribution**.

Common Probability Distribution

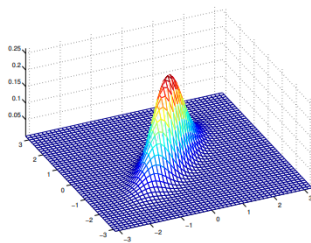
- Examples of multivariate normal distributions with zero mean $\mu \in \mathbb{R}^{2 \times 1}$ and different covariance matrices.



$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$



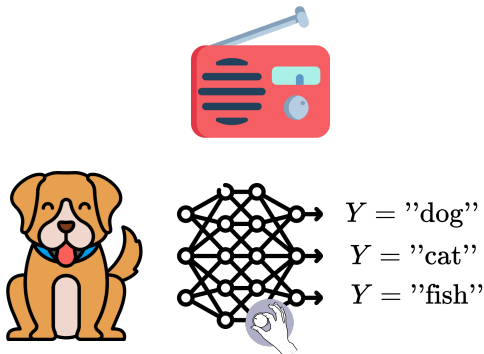
$$\Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$

Optimization

What is Optimization?

- **Optimization** comes from the same root as "optimal", which means "best".
- When you optimize something, you are "making it best".
- The word "best" is subjective and depends on context:
 - ▶ If you are a website administrator then you might want to minimize the amount of downtime
 - ▶ If you are a manufacturer of a product, you might want to minimize the production cost and maximize the profit
 - ▶ In the case of a neural network, we want to maximize the accuracy of a model (assuming the task of image classification) or minimize the error rate

What is Optimization?



- To listen to a radio you would be turning the knobs to catch the right frequency.
- Similarly, for neural networks, we will be adjusting the parameters (unknown) so that given an image of a dog, it assigns higher probability to the class "dog" and lower probabilities to all the other classes.

Optimization Preliminaries

- Considering that a neural network is nothing but a function $f(\mathbf{x})$, we denote it as the **objective function**. It is the function $f(\mathbf{x})$ that we want to either maximize or minimize (lets assume minimize).
- Let's say it is a function of n **variables**, $x_1, x_2, \dots, x_n$, ie, $f(x_1, x_2, \dots, x_n)$. So we want to "adjust" these unknown n variables as long as we find the "best" function.
- Sometimes, we want to put some constraints on how big or small some variables can be, denoting them by $c(\mathbf{x})$.
- Using these notations, we can write the optimization problem as:

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}), \text{ subject to } c(\mathbf{x}) \geq 1$$

- The outcome of the optimization are the optimal set of values for the unknown variables.

Outro

- Although in practice we will mostly be using already well implemented libraries and frameworks, it is still important to know these concepts for several reasons:
 1. To have a deeper understanding of what goes inside a neural network
 2. To become better deep learning researcher if you intend to become one
 3. To help you write better and more "optimized" code
 4. To help you debug code better and faster
 5. In job interviews at big tech companies they might ask you fundamental mathematical concepts and not being able to answer such questions is a big red flag
 6. Finally, a little bit of maths does not hurt
- In the next lecture, we will see how these concepts are being used in practice and learn about the foundations of deep learning.

Reading materials

- [Dive into Deep Learning](https://d2l.ai/index.html) (<https://d2l.ai/index.html>). Chapters: 1 and 2.
- [Deep Learning](https://www.deeplearningbook.org/) (<https://www.deeplearningbook.org/>). Chapters: Part I (2, and 3)